Canary Deployment

In Software Terms

You have a new version of your service ready. Instead of pushing it to all your servers at once, you deploy it to maybe 5% of your infrastructure. A small slice of real users start hitting the new version. The rest still use the old one.

You watch your metrics — error rates, response times, crash reports. If the new version behaves well, you gradually shift more traffic to it. 10%, then 25%, then 50%, then 100%. If something goes wrong, you just route all traffic back to the old version and barely anyone was affected.

What You Watch During a Canary

- Error rates going up?
- Response times getting slower?
- Users complaining more than usual?
- Memory or CPU spiking?

If any of those look bad, you roll back. Fast. With minimal damage.

Other common Deployment Strategy

- Rolling Deployment
- Blue-Green
- Feature Flags
- Big-Bang / Recreate

Strangler Fig Pattern

In Software Terms

You have an old system (a monolith, an old API, a legacy service). You don't want to rewrite it all at once — that's risky and expensive. Instead you build the new system piece by piece alongside it. New requests go to the new system. Old requests still hit the old system. Over time the new system handles more and more. Eventually the old system has nothing left to do and you shut it down.

The old system gets "strangled" slowly.

SAGA Pattern

The Problem First

In a monolith, when you do multiple database operations together, you wrap them in a **transaction**. Either everything succeeds, or everything rolls back. Clean and simple.

```
Begin Transaction
  - Deduct money from wallet
  - Create order
  - Reserve inventory
  - Send confirmation email
If anything fails → rollback everything
Commit
```

But in microservices, each service has its **own database**. You can't just wrap a transaction across 5 different databases owned by 5 different services. There's no single transaction manager sitting across all of them.

So what happens when step 3 fails after steps 1 and 2 already succeeded? You've taken money but didn't create the order. That's a mess.

SAGA solves this.

In Software Terms

You're building an e-commerce app. A user places an order. This touches 4 microservices.

```
Step 1 - Order Service       → Create order (status: pending)
Step 2 - Payment Service     → Deduct payment
Step 3 - Inventory Service   → Reserve the item
Step 4 - Delivery Service    → Schedule delivery
```

Everything goes well — great, order is confirmed.

But what if Step 3 fails (item is out of stock)?

```
Step 3 - Inventory Service   → FAILED (out of stock)

Now run compensating transactions backwards:

Undo Step 2 - Payment Service    → Refund the payment
Undo Step 1 - Order Service      → Cancel the order
```

You can't magically rollback like a database transaction. Instead you explicitly call undo operations on each service that already succeeded.

Two Ways to Implement SAGA

1. Choreography (No one is in charge)

Services talk to each other through events. Each service listens for an event, does its job, and fires the next event. No central brain.

```
Order Service creates order
    → fires "OrderCreated" event

Payment Service listens, deducts payment
    → fires "PaymentDone" event

Inventory Service listens, reserves item
    → fires "ItemReserved" event

Delivery Service listens, schedules delivery
    → fires "DeliveryScheduled" event
```

If inventory fails it fires "ItemReservationFailed" event and each service that already acted listens to that and runs its undo.

Think of it like a relay race. Each runner knows what to do when they receive the baton. No coach standing in the middle directing everyone.

Good — simple, no single point of failure, very decoupled.

Bad — hard to track what's happening across the whole flow. Debugging is painful.

2. Orchestration (One boss in charge)

A central service called the **Orchestrator** tells each service what to do, step by step. It knows the entire flow and manages failures.

```
Orchestrator says → Order Service: create order ✓
Orchestrator says → Payment Service: deduct payment ✓
Orchestrator says → Inventory Service: reserve item ✗ FAILED

Orchestrator says → Payment Service: refund payment
Orchestrator says → Order Service: cancel order
```

Think of it like a project manager. Team members don't talk to each other directly. Everything goes through the PM who knows the full picture and handles problems.

Good — easy to track, easy to debug, clear ownership of the flow.

Bad — orchestrator becomes a critical piece. If it goes down, nothing works.

Outbox Pattern

The Problem First

In microservices, after doing a database operation you often need to notify other services by publishing an event to a message broker like Kafka or RabbitMQ.

Sounds simple. But there's a dangerous gap.

```
Step 1 - Save order to database   ✓ success
Step 2 - Publish "OrderCreated"
         event to Kafka            ✗ FAILED
```

Your order is saved but nobody got notified. Inventory service never reserved the item. Payment service never got triggered. The event is just gone. Lost forever.

You can't wrap a database save and a Kafka publish in a single transaction. They're two completely different systems. This is the exact same cross-system consistency problem SAGA solves at a higher level — but here it's happening at a much smaller, lower level. Just one service, two systems.

Some people think — okay, what if I flip the order?

```
Step 1 - Publish event to Kafka   ✓ success
Step 2 - Save order to database   ✗ FAILED
```

Now even worse. The event fired but the order doesn't even exist in your database. Other services are reacting to a ghost order.

Either way you have a reliability problem. The Outbox pattern fixes this.

In Software Terms

Instead of publishing an event directly to Kafka after saving to the database, you save the event to a special table in the **same database** called the **outbox table**. Both the business data and the event get saved in a single database transaction.

```
Begin Database Transaction
  - Save order to Orders table
  - Save "OrderCreated" event to Outbox table
Commit  ← both happen together or neither does
```

Now a separate process called the **Message Relay** (the postman) constantly watches the outbox table. When it sees unpublished events, it picks them up and publishes them to Kafka. Then marks them as published.

```
Message Relay Process:
  - Reads unpublished events from Outbox table
  - Publishes to Kafka                          ✓
  - Marks event as published in Outbox table    ✓
```

Why This Works

The key insight is that saving to the outbox table and saving your business data happen in the **same database transaction**. Same database, same commit, same atomic operation. Either both succeed or neither does. The unreliable part (publishing to Kafka) is now handled separately by a dedicated process that can retry as many times as needed.

```
Without Outbox:
Save to DB  +  Publish to Kafka  =  Two separate operations, can fail independently

With Outbox:
Save to DB  +  Save to Outbox   =  One transaction, always consistent
                    ↓
              Message Relay publishes to Kafka (retries until success)
```

Load Balancer

In Software Terms

You have 5 instances of your Order Service running (because one server can't handle all the traffic alone). When 10,000 users hit your app at the same time, the load balancer sits in front of all 5 instances and distributes incoming requests across them so no single instance gets crushed.

```
User Requests
                    (10,000/sec)
                         ↓
                   Load Balancer
                  /   /   |   \   \
                 ↓   ↓    ↓    ↓   ↓
              Srv1 Srv2 Srv3 Srv4 Srv5
             (2000)(2000)(2000)(2000)(2000)
               requests each
```

Each server handles a manageable chunk. Everyone stays healthy.

What Else Load Balancer Does

Distributing traffic is just the most obvious job. It also does a few other important things.

Health Checking — it constantly pings each server asking "are you alive?" If a server crashes or becomes unhealthy, the load balancer stops sending traffic to it automatically. Users never even know a server went down.

```
Load Balancer pings every 10 seconds:
Srv1 → alive ✓  send traffic
Srv2 → alive ✓  send traffic
Srv3 → dead  ✗  stop sending traffic
Srv4 → alive ✓  send traffic
Srv5 → alive ✓  send traffic
```

SSL Termination — instead of every server handling the overhead of encrypting and decrypting HTTPS traffic, the load balancer does it once and passes plain HTTP to the servers internally. Less work for your servers.

Single Entry Point — users only ever know one address (your domain). They have no idea how many servers are behind it or what's happening internally.

How It Decides Where to Send Traffic

There are a few strategies called load balancing algorithms.

Round Robin — send request 1 to server 1, request 2 to server 2, request 3 to server 3, then back to server 1. Simple rotation. Like dealing cards.

Least Connections — send the next request to whichever server currently has the fewest active connections. Smarter than round robin because some requests take longer than others.

IP Hashing — always send the same user to the same server based on their IP address. Useful when a server stores session data locally and you need the same user to always land on the same server.

Weighted — some servers are more powerful than others. You tell the load balancer "server 1 can handle 3x more than server 2" and it distributes accordingly.

Types of Load Balancers

Layer 4 Load Balancer (Transport Level) — works at the network level. It just looks at IP addresses and ports and routes traffic without caring what's inside the request. Very fast, very simple.

Layer 7 Load Balancer (Application Level) — smarter. It actually looks inside the request — the URL, headers, cookies — and makes routing decisions based on that content.

For example a Layer 7 load balancer can say:

```
Requests to /api/payments  → route to Payment servers
Requests to /api/orders    → route to Order servers
Requests to /api/users     → route to User servers
```

Most modern systems use Layer 7 because it gives much more control.

Without vs With Load Balancer

```
Without Load Balancer:

1000 users → Single Server → Server explodes 💥
                             App is down
                             Everyone suffers


With Load Balancer:

1000 users → Load Balancer → Server 1 (333 users) ✓
                           → Server 2 (333 users) ✓
                           → Server 3 (334 users) ✓
                             All healthy, all serving
```

Popular Load Balancers

NGINX, AWS ALB / ELB, HAProxy, Traefik, Cloudflare